

L'Illusionista e il Motore

Cosa c'è davvero sotto Copilot e ChatGPT, e perché smette di valere l'idea di software che dà sempre la stessa risposta.

Il prodotto e il motore: Copilot come interfaccia, l'LLM come motore

- Il prodotto è tutto ciò che vedete: interfaccia, login aziendale, cronologia, connettori a SharePoint o Jira.
- Il motore è un modello linguistico: riceve testo, restituisce testo. Nient'altro.
- Lo stesso motore gira sotto prodotti diversi, e lo stesso prodotto può cambiare motore senza dirvelo.

DOVE SI SEPARANO

Interfaccia (Copilot)

▼ **prompt + contesto**

Orchestratore

▼ **chiamata API**

Motore LLM

Le regole aziendali, i permessi e i log stanno nei primi due strati, non nel terzo.

Cosa non è il motore: non un database, non «cerca» una risposta

- Nessuna tabella, nessun indice, nessuna query: nei pesi non è conservato il testo di partenza.
- Non c'è nemmeno una ricerca full-text: la domanda non viene confrontata con dei documenti.
- Se un dato non è nel prompt o non arriva da uno strumento esterno, il motore non lo legge: lo ricostruisce.
- Da qui le risposte formalmente perfette e sostanzialmente sbagliate: numeri di commessa, articoli di legge, nomi di colonne.

QUESTO NON ESISTE

```
SELECT risposta FROM conoscenza WHERE  
domanda = 'ferie residue';
```

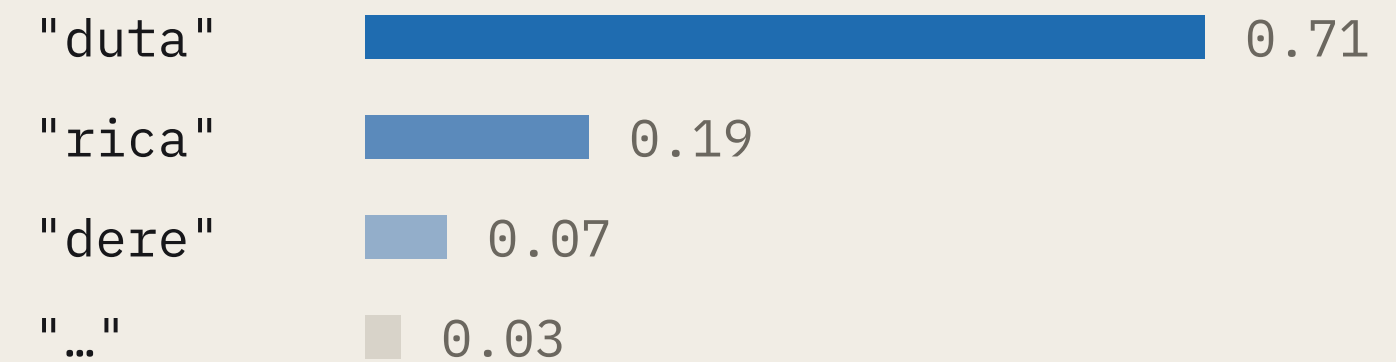
QUESTO SÌ

```
next_token(testo) → distribuzione di  
probabilità
```

Cosa fa davvero: previsione del prossimo pezzo di testo

- Input: tutto il testo fin qui. Output: un punteggio di probabilità per ogni possibile pezzo successivo.
- Se ne sceglie uno, lo si accoda all'input e si ripete: il testo esce un pezzo alla volta.
- Il modello non sa dove sta andando la frase: nessun piano, nessuna verifica a posteriori.

prompt: «La password è sca»



Dal determinismo alla probabilità: perché la stessa domanda può dare risposte diverse

SOFTWARE CHE CONOSCETE

```
calcolaIVA(100, 22) → 122.00 // sempre
```

Stesso input, stesso output. Il test è un confronto: `assertEquals`.

IL MOTORE

```
chiedi("riassumi l'ordine") → testo A, poi  
testo B, poi C
```

A ogni passo si *campiona* tra più token possibili: percorsi diversi, esiti diversi.

- La variabilità non è un bug: è il meccanismo di generazione. Si può ridurre, non annullare.
- Conseguenza operativa: si validano formato e vincoli della risposta, non la stringa esatta.

La temperatura: la leva che regola quella variabilità

- È un parametro della richiesta, non una proprietà del modello: riscalda le probabilità prima della scelta.
- Bassa: la distribuzione si concentra sul token più probabile. Alta: si appiattisce, e i token rari diventano giocabili.
- Non regola la correttezza: a temperatura 0 il motore sbaglia in modo ripetibile.

```
{ "prompt": "...", "temperature": 0.2, "top_p": 0.9 }
```

0 – 0.3 estrazione dati, classificazione, output JSON

0.4 – 0.7 redazione di testi, riformulazioni

0.8 – 1.2 brainstorming, varianti, idee

Verso il modulo 2: come sono fatti, internamente, i «pezzi di testo» che il motore prevede

IL PEZZO SI CHIAMA TOKEN — E NON È UNA PAROLA

SAP _M ES _ 202 4 «SAP_MES_2024» = 6 token

- I codici articolo, gli ID e le sigle si spezzano in modo poco intuitivo: è la radice di molti errori su dati strutturati.
- I token sono anche l'unità di misura del contesto e del costo: quanto testo entra, quanto se ne può tenere.
- Modulo 2 — Anatomia pratica del motore: da testo a token, da token a numeri, e come il motore li mette in relazione.